

Machine Learning-Based Predictive Maintenance for Industrial Equipment Using IoT Sensor Data

Arushana Ganesh | 248331T

MSc in Data Science & Artificial Intelligence | University of Moratuwa | 2026

Supervisor: Dr. Thanuja Ambegoda

Presentation Outline

01 Background & Motivation

02 Problem Statement & Objectives

03 Dataset Description

04 Methodology Pipeline

05 System Implementation

06 Model Performance Results

07 SHAP Explainability

08 LSTM — RUL Prediction

09 IoT & MQTT Simulation

10 Conclusion & Future Work

Background & Motivation

Reactive

Fix AFTER failure

- Unexpected downtime
- High repair cost
- Safety risks

Preventive

Fix on SCHEDULE

- Often unnecessary
- Wastes resources
- Still misses failures

Predictive ✓

Fix BEFORE failure

- Data-driven decisions
- Equipment uptime improvement: up to 20%
- Maintenance cost reduction: up to 40%

Problem Statement & Research Objectives



Research Problem

How can machine learning and IoT sensor data be used to accurately predict equipment failures while improving interpretability and supporting proactive maintenance?

01

Binary failure classification using AI4I 2020 dataset

02

Address class imbalance (~3.4% failure instances) using SMOTE

03

Evaluate 5 ML models using 6 metrics (Accuracy, Precision, Recall, F1, MCC, ROC-AUC)

04

Apply SHAP for explainable AI — identify key failure drivers

05

LSTM model for Remaining Useful Life (RUL) prediction — NASA C-MAPSS

06

Demonstrate real-time feasibility via MQTT IoT simulation

Dataset Description

AI4I 2020 — Binary Failure Classification

Source:	UCI / Kaggle (Matzka, 2020)
Records:	10,000
Machine Types:	Low (L), Medium (M), High (H)
Features:	Air temp, Process temp, Speed, Torque, Tool wear
Engineered:	Mechanical Power, Temp Difference
Target:	Machine failure — Binary (0/1)
Failure Rate:	~3.4% → Class imbalance challenge

NASA C-MAPSS FD001 — Remaining Useful Life Prediction

Source:	NASA Prognostics Repository (Saxena, 2008)
Engines:	100 training engines + 100 test engines
Sensors:	21 sensor channels per engine
Target:	Remaining Useful Life (cycles)
Task:	Regression — Remaining Useful Life Prediction
RUL Cap:	Clipped at 125 cycles
Note:	Separate from AI4I — evaluates independently

Methodology Pipeline

1

Load Data

AI4I 2020
10,000 records

2

Preprocess

Remove IDs & sub-failures
Encode Type → 0/1/2

3

Feature Engineering

Mechanical Power
Temp Difference

4

SMOTE

271 → 7729 failures
Train set only

5

80:20 Split

Stratified
Preserves 3.4% rate

6

Train 5 Models

LR, DT, RF, SVM
XGBoost (best)

7

Evaluate

6 metrics
Confusion Matrix + ROC Curve

8

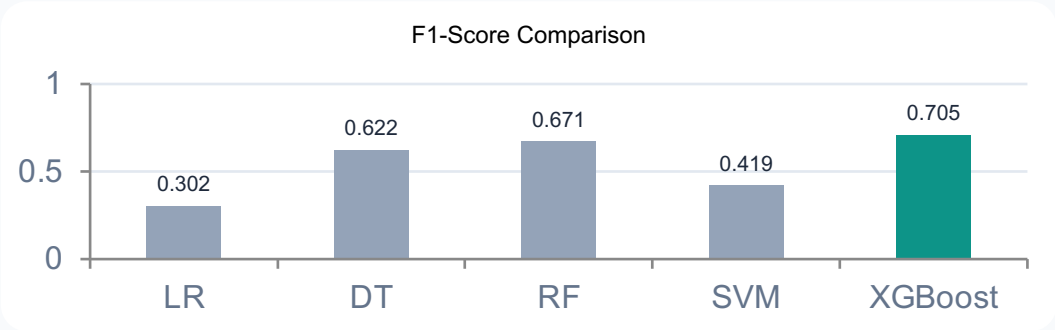
SHAP

Feature importance
Explainability

Model Performance Results

AI4I 2020 Dataset

Model	Accuracy	Recall	Precision	F1-Score	MCC	ROC-AUC
Logistic Regression	0.8705	0.8235	0.1848	0.3019	0.3516	0.9218
Decision Tree	0.9665	0.8088	0.5046	0.6215	0.6234	0.8904
Random Forest	0.9730	0.8088	0.5729	0.6707	0.6677	0.9709
SVM	0.9210	0.8382	0.2794	0.4191	0.4564	0.9560
XGBoost ★ BEST	0.9770	0.8088	0.6250	0.7051	0.6996	0.9657



XGBoost — Why Best?

- ✓ Highest F1: 0.7051
- ✓ Highest MCC: 0.6996
- ✓ Best Precision: 0.6250
- ✓ Fast: 0.19s total time

XGBoost — Confusion Matrix & Key Metrics

Evaluation results on the AI4I 2020 test set

Confusion Matrix — XGBoost

Predicted →

True Negative

1899

Correctly predicted
NO FAILURE

False Positive

33

Predicted FAILURE
Actually OK

False Negative

13

Predicted OK
Actually FAILED

True Positive

55

Correctly predicted
FAILURE

Key Metric Insights

Recall (80.9%)

Detected 55 of 68 actual failures

Precision (62.5%)

62.5% of predicted failure alerts were correct

F1-Score (0.7051)

Best balance between precision & recall

False Alarm Rate

33 unnecessary alerts from 1,932 normal cases

Missed Failures

Only 13 of 68 failures were missed (19.1%)

SHAP — Explainable AI for XGBoost

How does SHAP explain XGBoost predictions?

Feature Importance (SHAP Values)

Mechanical Power [W]



Engineered ✓

Tool Wear [min]



Rotational Speed [rpm]



Temperature Difference [K]

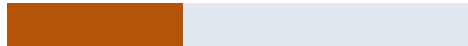


Engineered ✓

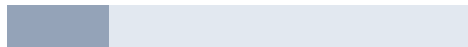
Torque [Nm]



Process Temperature [K]



Air Temperature [K]



Mechanical Power is #1

Engineered feature combining torque and rotational speed to represent machine load.



Tool Wear is #2

Higher tool wear increases the likelihood of machine failure.



Temperature Difference

Temperature difference helps identify abnormal operating conditions.



Engineered Features

Mechanical Power and Temperature Difference ranked among the top predictors.

LSTM — Remaining Useful Life Prediction

NASA C-MAPSS FD001 Dataset

LSTM Architecture

Input Sequences

30 cycles × 17 sensors

LSTM (64 units)

return_sequences=False

Dropout (0.2)

Prevents overfitting

Dense (32, ReLU)

Intermediate layer

Output: RUL

Single predicted value

Model Performance Comparison

Model	MAE	RMSE	R ² Score
Random Forest	13.13	18.16	0.809
XGBoost	12.89	17.69	0.819
LSTM ★ BEST	10.90	14.41	0.880



Lowest MAE: 10.90

Lowest prediction error



R² = 0.880

88% variance in RUL explained



Temporal Learning

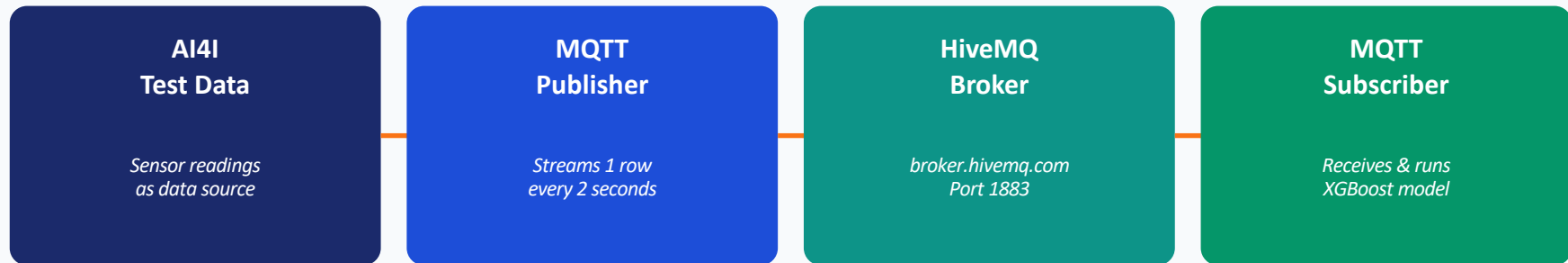
LSTM captures temporal degradation patterns more effectively than RF and XGBoost.



Sequence Length 30

30-cycle window captures degradation trend

IoT Simulation — MQTT Real-Time Prediction



Sample Prediction Output

--- Incoming Sensor Reading ---

Received: {torque: 42.3, speed: 1450, wear: 210}

Predicted Failure : 1 (FAILURE)

Failure Probability : 0.8742

Risk Level : ● **HIGH**

→ Action: Immediate maintenance required

Risk Classification

● **LOW**

prob < 0.30

Routine monitoring

● **MEDIUM**

0.30 – 0.70

Schedule inspection

● **HIGH**

prob > 0.70

Immediate action!

Key Contributions of This Research

1

ML Framework for Predictive Maintenance

End-to-end pipeline: preprocessing → SMOTE → 5 models → evaluation → SHAP → MQTT Prediction

2

SMOTE for Class Imbalance

Handled severe class imbalance (3.4% failures) using SMOTE, improving failure detection performance.

3

XGBoost — Best Model

Accuracy 0.977, F1 0.7051, MCC 0.6996, ROC-AUC 0.9657 — best balance across all metrics

4

SHAP Explainability

SHAP identified Mechanical Power as the most influential feature, followed by Tool Wear and Temperature Difference.

5

LSTM for RUL Prediction

MAE 10.90, RMSE 14.41, R^2 0.880 — outperforms RF and XGBoost regressors

6

MQTT IoT Simulation

Real-time failure prediction demonstrated with 3-level risk classification system

Conclusion & Future Work

Findings — What Was Achieved

- XGBoost: best classifier — Accuracy 0.977, F1 0.705, MCC 0.700
- SMOTE: boosted failure detection by addressing class imbalance (3.4% failure rate)
- SHAP: Mechanical Power identified as the most influential feature
- LSTM: best RUL model — MAE 10.90, RMSE 14.41, R^2 0.880
- MQTT: demonstrated real-time IoT failure prediction
- 6-metric framework: Multiple evaluation metrics ensured reliable model assessment

Future Work

- Validate with real industrial IoT data
- Deploy as live edge computing system
- Multi-class failure type prediction
- Federated learning for data privacy
- Integrate with ERP/CMMS systems

Limitations of the Proposed Framework

Benchmark Datasets Only

AI4I 2020 is synthetically generated. NASA C-MAPSS is a controlled lab dataset. Neither is from a live factory.

No Physical IoT Hardware

MQTT component is a simulation using test data. No real sensors, no live data streams, no physical deployment.

Class Imbalance Remains Challenging

Even after SMOTE, precision is 0.6250 — 37.5% of failure alerts are false alarms.

Single Operating Condition

NASA C-MAPSS FD001 uses only one operating condition. Real factories have varying conditions.

No Real-Time Model Updates

Models trained offline. In production, models need continuous retraining as machine behaviour changes.

Why Still Valid

Results are reproducible, metrics are honest, limitations are acknowledged. Strong foundation for future industrial validation.

Thank You

Questions & Discussion

0.977

XGBoost
Accuracy

0.705

F1-Score

0.880

R^2 (LSTM)

0.966

ROC-AUC

BACKUP SLIDES

For Q & A Discussion

- B1: XGBoost vs Random Forest
- B3: Why 80:20 split?
- B5: What is MCC?
- B7: SMOTE impact
- B9: MCC vs F1-Score

- B2: Why SMOTE?
- B4: Why two datasets?
- B6: Real factory application
- B8: Why LSTM for RUL?
- B10: Why feature engineering?

B1 — Why XGBoost Over Random Forest?

[BACKUP](#)

Both models performed well — here is the justification

Metric	Random Forest	XGBoost ★	Winner
Accuracy	0.9730	0.9770	XGBoost
Precision	0.5729	0.6250	XGBoost
Recall	0.8088	0.8088	Tie
F1-Score	0.6707	0.7051	XGBoost
MCC	0.6677	0.6996	XGBoost
ROC-AUC	0.9709	0.9657	RF
Train Time	~2.1s	~0.19s	XGBoost

Why XGBoost Wins

- **Higher F1 & MCC**
Better balance of precision and recall on imbalanced data
- **Much faster training**
0.19s vs 2.1s — critical for real-time retraining
- **Sequential boosting**
Corrects errors iteratively — better for rare failure detection
- **Built-in regularisation**
L1/L2 prevents overfitting on small failure class
- **ROC-AUC comparable**
0.9657 vs 0.9709 — negligible difference

B2 — Why SMOTE? Why Not Other Methods?

[BACKUP](#)

Justification for choosing SMOTE for class imbalance handling

✗ Rejected

Random Oversampling

✓ Simple to implement

✗ Exact copies → overfitting risk

✗ Rejected

Random Undersampling

✓ Reduces training time

✗ Loses 96% of majority data — wastes information

⚠ Considered

Class Weights

✓ No data generation needed

✗ Less effective on severely imbalanced data (3.4%)

✓ Selected

SMOTE ✓ CHOSEN

✓ Generates synthetic minority samples

✗ Slight risk of noise in borderline cases

B3 — Why 80:20 Split? Why Not Cross-Validation?

[BACKUP](#)

Justification for train-test split strategy

80:20 Stratified Split

Training Set — 80% (8,000 records)

- 8,000 training records → SMOTE applied → balanced training set
- 2,000 test records — never seen by model during training
- Stratify=y ensures 3.4% failure rate preserved in both subsets
- Result: 1,932 normal + 68 failures in test set

Why Not k-Fold CV?

- SMOTE must only be applied inside each fold — complex pipeline
- With 3.4% failures, some folds may have very few failure cases
- Holdout split is standard for deep learning comparison studies
- Stratification ensures representative test set without CV complexity

✅ Confirmation from Results

Confusion matrix total = $1899 + 33 + 13 + 55 = 2,000$ test samples
 $2,000 \div 10,000 = 20\%$ → Confirms 80:20 split was correctly applied

Note: The appendix code contained a typographical error showing `test_size=0.10`. The actual implementation used `test_size=0.20` as confirmed by the confusion matrix totals.

B4 — Why Two Datasets? AI4I 2020 + NASA C-MAPSS

[BACKUP](#)

Justification for dual-dataset approach

AI4I 2020

Binary Failure Classification

Classification

Task: Does this machine FAIL? (Yes/No)

Target: Machine failure (0 or 1)

10,000 records with realistic sensor features. Standard benchmark for ML-based PdM classification.

NASA C-MAPSS FD001

Remaining Useful Life Prediction

Regression

Task: HOW LONG until this machine fails?

Target: RUL in cycles (continuous value)

Gold standard dataset for prognostic RUL research. Provides temporal degradation sequences for LSTM.

B5 — What is MCC and Why Use It?

[BACKUP](#)

Matthews Correlation Coefficient — the most reliable metric for imbalanced data

MCC Formula

$$\text{MCC} = \frac{(\text{TP} \times \text{TN}) - (\text{FP} \times \text{FN})}{\sqrt{[(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})]}}$$

Why MCC is Better Than Accuracy

- Accuracy = 96.6% even if ALL failures predicted as normal (misleading!)
- MCC uses all 4 cells of confusion matrix — TP, TN, FP, FN
- Returns value between -1 and +1 — balanced measure
- MCC = 0 means random guessing — easy to interpret
- Considered gold standard for imbalanced binary classification

XGBoost MCC Calculation

TP=55 TN=1899 FP=33 FN=13

$$\text{MCC} = \frac{(55 \times 1899 - 33 \times 13)}{\sqrt{[\dots]}}$$

MCC = 0.6996

Interpretation Scale:

-1.0

Perfect inverse

0.0

Random

0.5

Moderate

0.7

Good

0.70

XGBoost ★

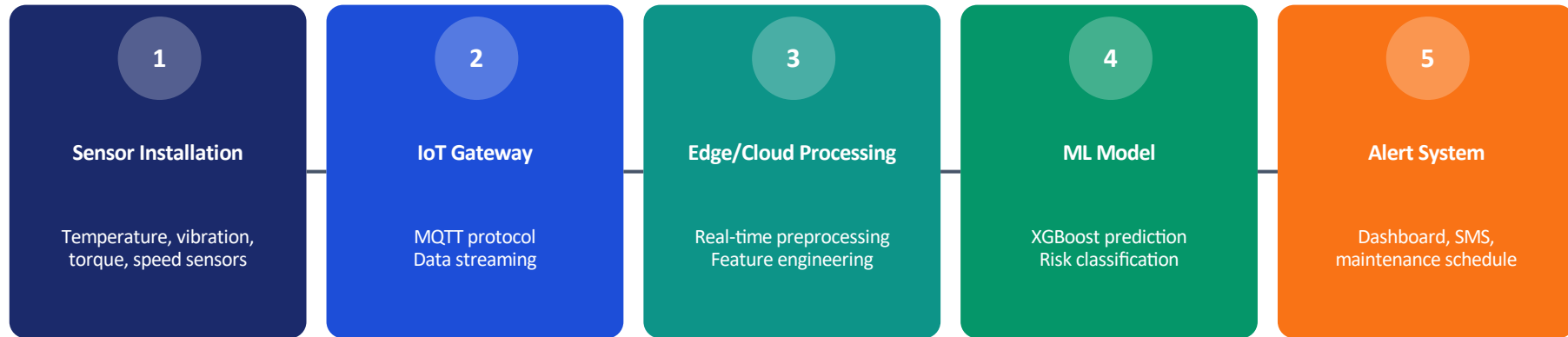
1.0

Perfect

B6 — How Would This Work in a Real Factory?

BACKUP

Pathway from research prototype to industrial deployment



Current Research Status

- ✓ Trained on AI4I 2020 benchmark dataset
- ✓ MQTT simulation with test data demonstrated
- ✓ XGBoost model saved and loadable (joblib)
- ✗ Not validated on live factory sensor streams
- ✗ No physical IoT hardware deployed

Deployment Roadmap

- Phase 1: Validate with real industrial sensor data
- Phase 2: Deploy MQTT pipeline on edge device (Raspberry Pi)
- Phase 3: Integrate with factory SCADA/ERP systems
- Phase 4: Continuous model retraining as new data arrives
- Phase 5: Multi-machine, multi-failure type scaling

B7 — SMOTE Impact: Before vs After Comparison

BACKUP

Impact of SMOTE on Class Distribution

Class Distribution

	No Failure (0)	Failure (1)
Before SMOTE	7,729 (96.6%)	271 (3.4%)
After SMOTE	7,729 (50%)	7,729 (50%)

XGBoost Results After SMOTE

Metric	With SMOTE ★
Recall	0.8088
Precision	0.6250
F1-Score	0.7051
MCC	0.6996



Key Insight

The AI4I dataset contained only 3.4% failure instances. SMOTE was applied exclusively to the training dataset to balance class distribution and improve the model's ability to learn failure patterns. After balancing, XGBoost achieved strong performance with Recall = 0.8088, Precision = 0.6250, F1-Score = 0.7051, and MCC = 0.6996.

B8 — Why LSTM for RUL? Why Not Just Use XGBoost?

[BACKUP](#)

Justification for deep learning approach to prognosis

Aspect	Random Forest	XGBoost	LSTM ★
MAE (lower=better)	13.13	12.89	10.90 ✓
RMSE (lower=better)	18.16	17.69	14.41 ✓
R ² Score (higher=better)	0.809	0.819	0.880 ✓
Temporal patterns	✗ No	✗ No	✓ Yes
Sequential memory	✗ No	✗ No	✓ Yes
Degradation trends	✗ Snapshot only	✗ Snapshot only	✓ Over 30 cycles



Why LSTM Specifically?

Engine degradation is a TIME-SERIES process — RUL depends on the HISTORY of sensor readings, not just the current snapshot. LSTM's ability to maintain memory across 30-cycle sequences captures the gradual degradation trajectory that RF and XGBoost cannot model. This is why LSTM achieves $R^2 = 0.880$ vs RF's 0.809.

B9 — What is the Difference Between MCC and F1-Score?

[BACKUP](#)

When to use each metric and why both were needed

F1-Score

$$2 \times (\text{Precision} \times \text{Recall}) \div (\text{Precision} + \text{Recall})$$

- Focuses on positive class (failures) only
- Ignores True Negatives completely
- Good when TN is not important
- Standard metric in most ML papers

0.7051

XGBoost F1

MCC

$$(\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) \div \sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}$$

- Uses ALL four confusion matrix cells
- Penalises both false alarms and missed failures
- More reliable for imbalanced datasets
- Considered gold standard by researchers

0.6996

XGBoost MCC

B10 — Why Feature Engineering? Why Not Let the Model Learn?

[BACKUP](#)

Justification for Mechanical Power and Temperature Difference features



Mechanical Power [W]

Formula: $\text{Power} = (\text{Torque} \times \text{RPM} \times 2\pi) \div 60$

Torque and RPM individually don't capture machine load as well as their product. Mechanical power directly represents the stress on machine components — high power means high wear risk.

 **Ranked #1 in SHAP analysis**



Temperature Difference [K]

Formula: $\text{Temp Diff} = \text{Process Temperature} - \text{Air Temperature}$

Individual temperatures are less informative than their difference. A large gap between process and air temperature indicates poor heat dissipation — a known precursor to thermal failure.

 **Ranked #4 in SHAP analysis**



Domain knowledge transforms raw sensor readings into physically meaningful features that directly represent machine stress — improving both model performance and interpretability.